EC551 – Advanced Digital Design with Verilog and FPGAs
Prof. Douglas Densmore
Fall 2026
Exam 1
Date: 10/15/26

Name:

BU ID:

1. The exam consists of **5 major questions over 26 pages**. **Make sure you have all 26 pages.**
2. **You have 24 hours to do this exam**. **It is due back on 10/16/26 by 1:30pm EST to Gradescope.**
3. The maximum grade is **145 points**.
4. Please write your name and BU ID on **all pages you submit.**
5. The solutions should be written in the spaces provided or on separate pieces of paper clearly labeled.
6. If you need to use extra sheets of paper, write your name and BU ID on them and clearly indicate the question number.
7. This is an open book, open note exam. **However, you should NOT be discussing ANYTHING with other students. If you have any questions, you should post a Piazza question and Prof. Densmore will answer as quickly as possible.**
8. If you are found to have extensively used the Internet (found complete solutions, copied whole sections of Verilog code, **used AI**, etc.) you will receive a 0 on the exam and face disciplinary action. **If you have any questions, please ask in advance.**
9. Boston University Academic Code of Conduct applies.

Problem	Possible Points	Your Score
1	45	
2	55	
3	20	
4	10	
5	15	
Total	145	

Problem 1 (45 points): Combinational Logic Design

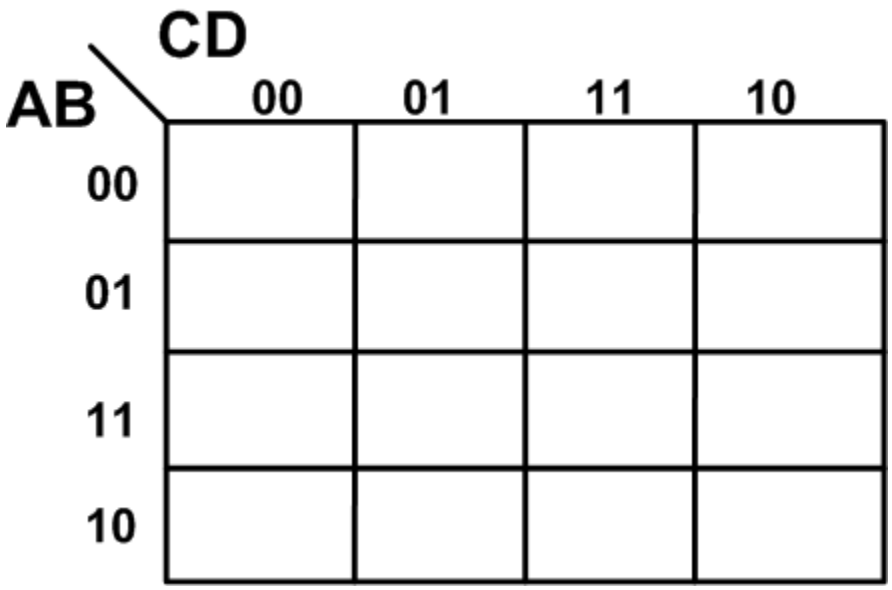
F (a, b, c, d) = a'b'c'd' + ab'c'd' + b'cd' + bc'd + bcd + a'bd

a. **Finish filling in the truth table for this expression. It is a completely specified function. All blank fields need to be filled in. (1/4 pt per blank; 5pts total) Hint: doing part C's K-map first might help.**

abcd value	Input				????
	????	b	c	????	f
0	0	????	0	0	1
1	0	0	0	1	0
2	0	0	????	0	????
3	????	0	1	1	0
4	0	????	0	0	0
5	0	1	0	1	????
6	0	????	1	????	0
7	0	1	1	1	1
8	1	0	0	0	????
9	1	????	0	1	0
10	????	0	1	????	1
11	1	0	????	1	0
12	1	1	0	0	0
13	1	1	0	????	????
14	1	1	1	0	0
15	1	????	1	????	1

b. **What is the Σm expression for the inverse of this equation? What is the ∏M expression for this function? (1pt each; 2pts total)**

c. Fill out the following K-map for the equation. In addition, fill out the following table used for Quine McClusky for the equation. Use both of these to create a minimal SOP expression. Show your work. There may be more rows than you need in the table. The column count for the minterms is correct though. (4pts each; 8pts total)



Minterms (please list them in "order" from left to right)								
PIs	**a'b'c'd'**							**abcd**

Minimal Expression = F (a, b, c, d) =

d. Highlight the Essential Prime Implicants in the Quine McClusky table. Explain why these are EPIs. (3pts total)

e. Are there any static hazards in your circuit? If so, what type? Explain and fix if appropriate. If fixed, provide the new SOP expression. (4pts total)

f. Draw the circuit for your minimized SOP style circuit. (2pts total)

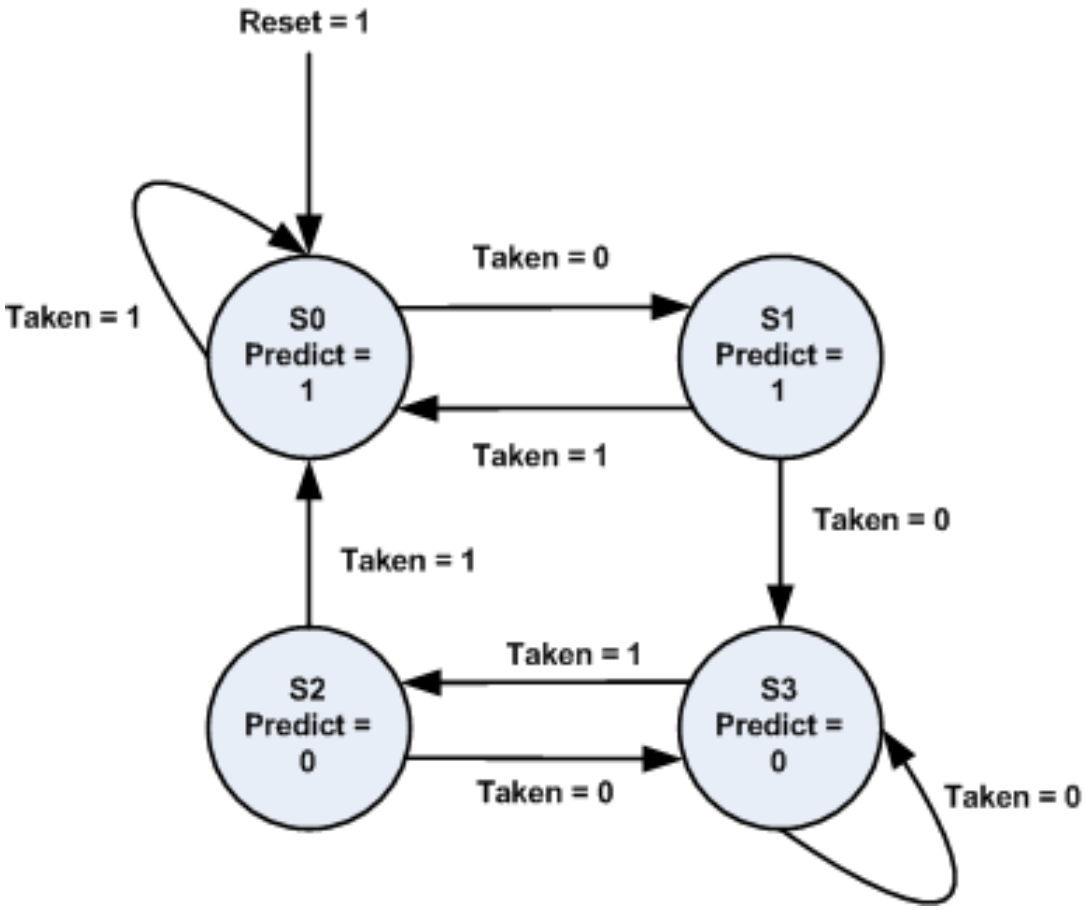
g. Create a structural Verilog description for the circuit you drew in Part f. Explain in one sentence why it is a structural description. (3pts Verilog, 2pts explanation; 5pts total) Feel free to attach a separate sheet if you type it up.

h. Create a behavioral Verilog description for the circuit you drew in Part f using a continuous assignment statement. Explain in one sentence why it is a behavioral description. (5pts Verilog, 2pts explanation; 7pts total) Feel free to attach a separate sheet if you type it up.

i. **Provide the always block statements separately to implement this function instead of continuous assignment. (3pts Verilog, 2pts explanation; 5pts total)**

j. What could you have done in problem h to make the Verilog unsynthesizble? Do NOT use delay statements or syntax errors as answers. (4 pts total)

Problem 2 (55 points): Sequential Logic Design



a. Complete the truth table for this FSM. In the interest of simplicity "Reset" is not listed. (10pts total)

Taken	cs1	cs0	ns1	ns0	Predict
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

b. Draw the controller diagram for this FSM. The next state and output combinational logic should be drawn as a black box with the Boolean equations written on it in lieu of drawing the circuit. Feel free to determine the combinational logic equations however you see fit. Be aware that maximal points will be for minimized POS expressions. Please include all the signals that appear in the truth table. (10pts total)

c. Create a behavioral Verilog description for the FSM using the template provided. (10pts total)

```
module state_machine (clk, reset, taken, predict);
        input clk, reset, taken;
        output predict;
        reg [?] ?; //use bit width and number of variables as desired

        //Fill in….
        ….
endmodule
```

d. How else could you have written the code in part c? Give 3 specific examples and explain the tradeoffs. (12pts total)

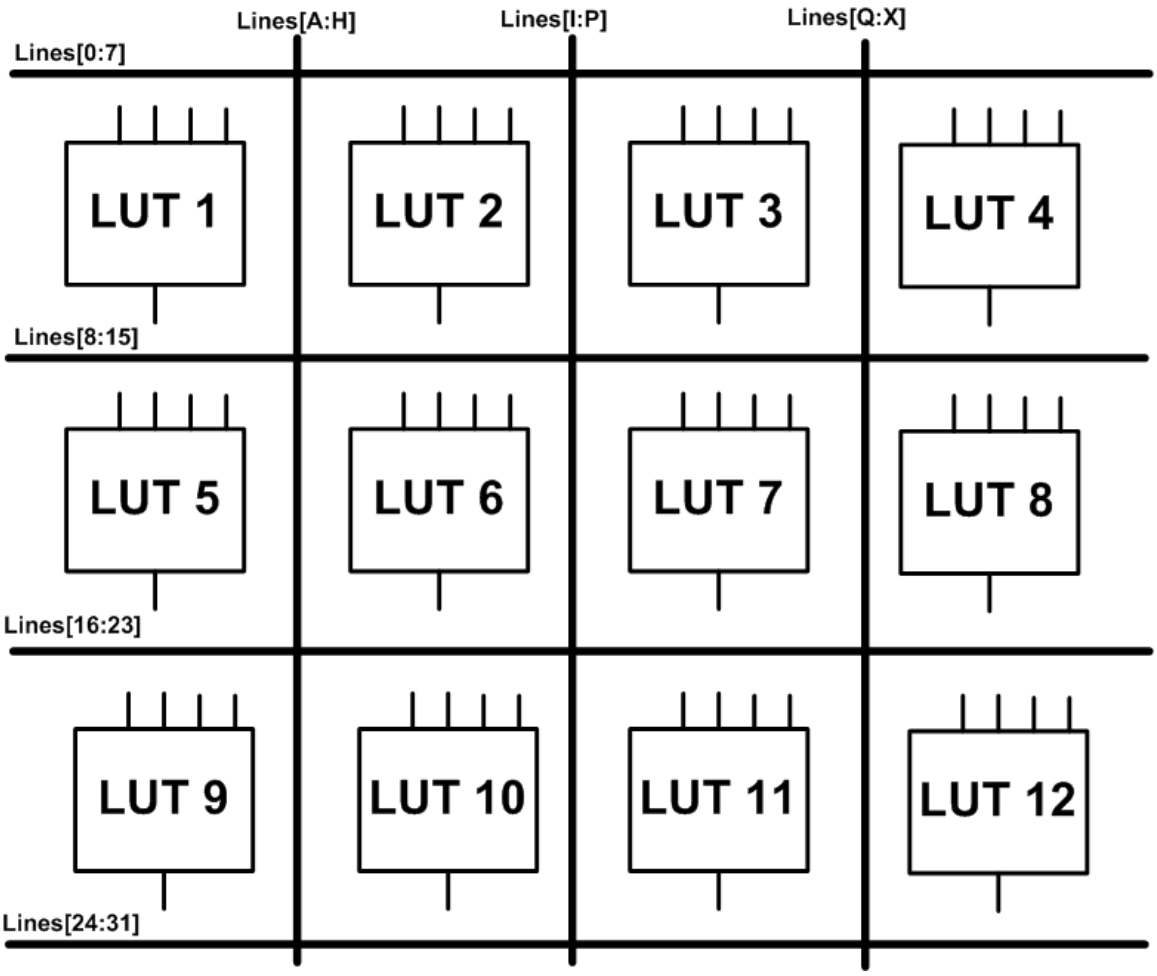
e. Reduce the number of states for the state table below using either the partitioning method or an implication chart. (10pts total)

		Present Input x	
		0	**1**
Present State	**A**	**D, 1**	**D, 1**
	B	**C, 1**	**C, 0**
	C	**B, 1**	**B, 1**
	D	**A, 1**	**C, 0**

f. Draw the resulting state transition graph and explain why this is a Mealy or Moore machine. (3pts total)

Problem 3 (20 points): FPGA Design

a. **A mock up FPGA topology is given below. Configure it for the following function: ((AB+C')(D+(E ⊕ F)))' – DO NOT SIMPLIFY!**
Make all needed connections and assigning LUT values. (15 pts)



LUT Gates Assignments - 4 inputs on top of each LUT, 1 output on the bottom.
Examples: AND2, NOT, XOR3 **(Feel free to write on the diagram when you have your FINAL answers).**

LUT 1
LUT 2
LUT 3
LUT 4
LUT 5
LUT 6
LUT 7
LUT 8
LUT 9
LUT 10
LUT 11
LUT 12

Horizontal Lines – LUT Input/Output Lines
Examples: Line 7 - LUT 1 IN0, Line 0 - A, Line 8 - LUT 2 OUT

Vertical Lines – Line to Line Connections
Examples: Line A – Line 1, Line Q – Line 15

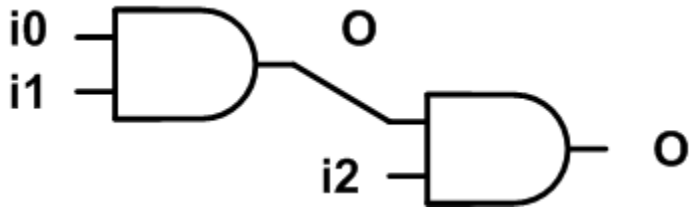
b. If each LUT could implement all 4-input functions and all possible line to line connections could be made, how much SRAM would be needed to store function and connection info. State your assumptions. (5pts)

Problem 4 (10 points): Multiple Choice and Paper Questions

Examine the provided Verilog code snippets. **Fix the provided block diagrams to correctly reflect this Verilog code.** **One error per block diagram.**

Snippet #1

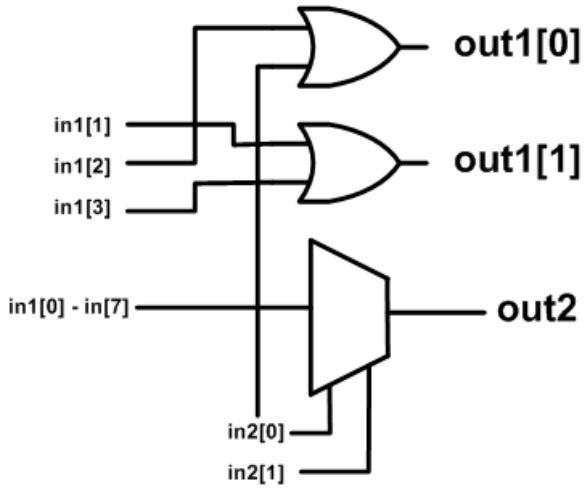
```
module AND3 (i0, i1, i2, o);
     input i0, i1, i2;
     output 0;
     wire temp;
     AND a0 (i0, i1, temp);
     AND a1 (i2, temp, 0);
endmodule
```



Snippet #2

```
module multi2 (in1, in2, out1, out2);
          input [7:0] in1;
          input [1:0] in2;
          output [1:0] out1
          output out2;

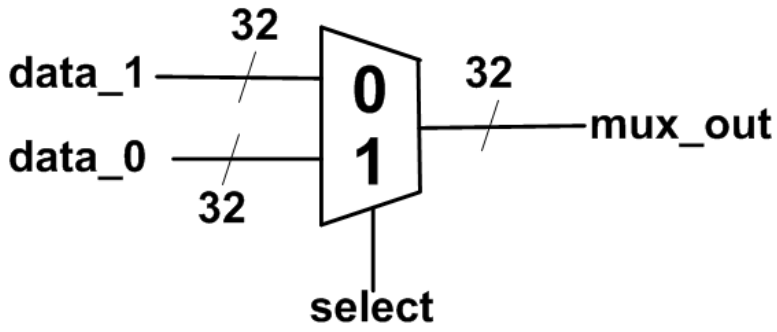
          assign out1 = (in1 >> 2) | in2;
          assign out2 = in1[in2];
endmodule
```



Snippet #3

```
module Mux_2_32 (mux-out, data_1, data_0, select);
     parameter word_size = 32;
     output [word_size - 1: 0] mux_out;
     input  [word_size - 1: 0] data_1, data_0;
     input  select;

     assign mux_out = select?data_1:data0;
endmodule
```

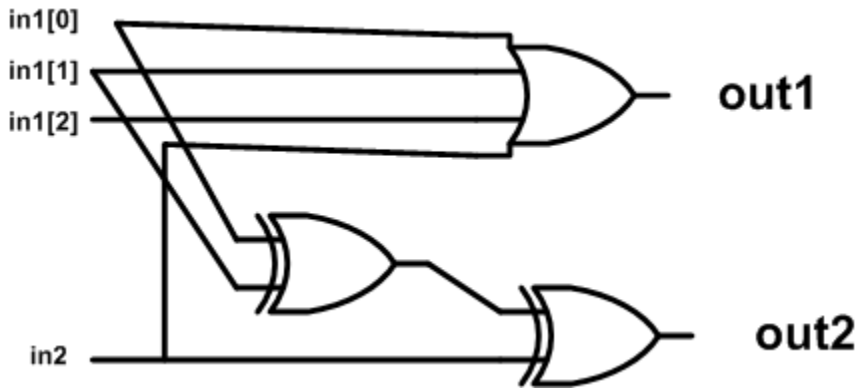


Snippet #4

```
module UnaryOperators (in1, in2, out1, out2);
     input [2:0] in1;
     input in2;
     output out1, out2;

     wire out1; out2;

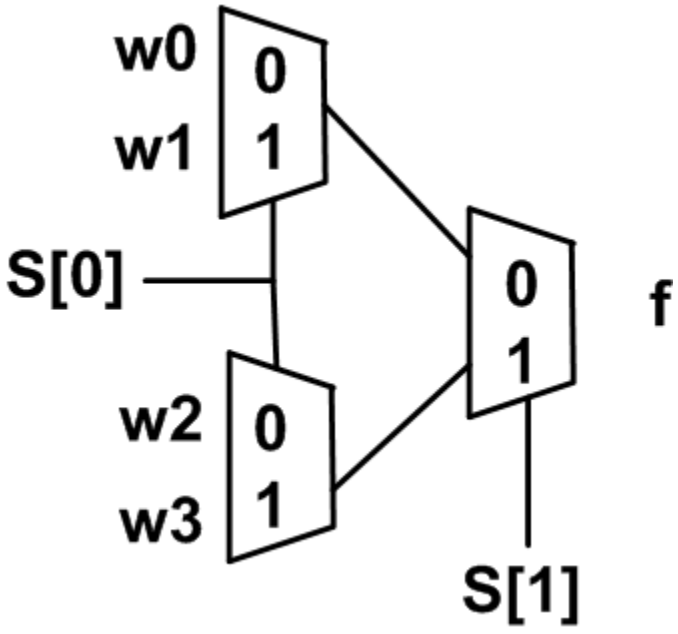
     assign out1 = |{in1, in2};
     assign out2 = ^in1;
endmodule
```



Snippet #5

```
module mux4to1 (w0, w1, w2, w3, S, f);
     input w0, w1, w2, w3;
     input [1:0] S;
     output f;

assign f = S[1] ? (S[0] ? w3 : w2) : (S[0] ? w1 : w0);
endmodule
```



“The Synthesis of Cyclic Combinational Circuits” (the paper we read in class)

What is the “technology independent” phase of logic synthesis? What is the cost metric used for a circuit? Describe substitution. Why would you want to get rid of cycles in combinational circuits? What is topological vs. electric feedback? (1pt each part; 5pts total)

Problem 5 (15 points): Multiple Choice **(1pt each; 10 pts total)**

a. **Verilog testbenches should be written so that they can be synthesized along with the DUT.**
   a. **True b. False**

b. **Always blocks are only used for sequential logic design.**
   a. **True b. False**

c. **When writing Verilog code, it is important to know about the hardware you are targeting to get the best synthesis result.**
   a. **True b. False**

d. **FPGAs that are larger will always require more power but will allow you to have more complex designs.**
   a. **True b. False**

e. **Non-blocking statements are better for modeling hardware. If you are going to synthesize a design you shouldn't use blocking statements.**
   a. **True b. False**

f. **Fix the following 5 Verilog code segments. These changes could be syntactic or functional. In the event you see multiple errors, choose the most functionally relevant (or mention all). DO NOT just fix random things; multiple wrong guesses will be penalized. Minimally find one error per question.**

F1.

```
//Do not want an accidental synthesis of a latch
always@(posedge clk)
    casez (op)
            2'b1? : // assignment block 1
            2'b?1:  // assignment block 2
    endcase
```

F2.

```
wire A, B, C
always @ (posedge  clk)
    begin
            A=C;
            B=A;
            C=B;
    end
```

F3.

```
Module foo (s, A, B, C, Z);
    input A, B, C;
    output Z;
    input [1:0] s;
always @ (s or A or B or C)
    case (s)
        3'b100 : Z<= A;
        3'b010 : Z<= B;
        3'b001 : Z<= C;
    endcase
endmodule
```

F4.

```
    Module adder_4_bit (A, B, C_in, Sum, C_out);
        Input [3:0] A, B;
        Input C_in;
        Output [3:0] Sum;
        Output C_out;
        Wire [3:0] A, B, C_in, Sum, C_out;

        fulladder FA3(A[3], B[3], C2, Sum[3], C_out);
        fulladder FA2(A[1], B[2], C_in, Sum[2], C2);
        fulladder FA1(A[2], B[1], C0, Sum[1], C1);
        fulladder FA0(A[0], B[0], C1, Sum[0], C0);
    endmodule
```

F5.

```
    //Should be a truly combinational circuit
    always @ (x or y)
        begin
            w = y & z;
            u = w | x;
        end
```

g. There is a group that is planning on creating a driving simulation game for their project.
   a. True/False

h. Using the "reg" syntax must lead to a register being synthesized.
   a. True/False

i. Circle the correct ordering of the digital design flow.

Design spec -> simulation -> design entry -> design partition -> synthesize

Design spec -> design entry -> design partition -> simulation -> synthesize

Design spec -> design partition -> design entry -> simulation -> synthesize

Design partition -> design spec -> design entry -> simulation -> synthesize

j. Verilog uses "Event Driven Simulation".
   a. True/False

k. How many functions can a 4-input LUT represent? (Circle one; 1pt)

16 4

Infinite 65536

l. We read two papers in class related to cyclic combinational logic circuits. What were each about (2-3 sentence for each paper) (1 pts each paper; 2 total)? Provide two questions you made for each paper for the paper game. (1 pt each question; 4pts total)

m. What are suggestions you have for improving the class? (1pt)